

Compass

Architecture & Feature Reference

PHASE 1

3 October 2026 · Manish Kumar

An agentic workspace on Azure: one Angular front end, one FastAPI backend, six modules behind a single sign-in. This document covers the architecture, every feature, where the business return comes from, where each piece of data lives, and all 136 endpoints.

Contents

Fourteen sections: the system at a glance, each module in turn, where the value and the data sit, and every endpoint.

Contents	2
At a glance: the whole system	3
The Azure infrastructure	4
Sign-in and the request path	5
Home (Chat)	7
Code (Agent Console)	8
Design	9
Pipelines	10
Estimate	11
Missions	12
Where the business value is	13
Where every piece of data lives	14
Azure Cosmos DB — database compass	14
Azure Blob Storage	14
Still on local disk, by design	14
The retention rule	15
API reference	16
Auth — compass/common/auth.py	16
Home — compass/home/routes.py, prefix /v1/chat	16
Code — compass/code/routes.py	17
Design — compass/design/routes.py	18
Pipelines — compass/pipelines/routes.py	18
Estimate — compass/estimate/routes.py	19
Missions — compass/missions/routes.py, prefix /v1/missions	19
Shared — compass/common/routes.py	20
Identity, ownership and deletion	21
For a new developer	22
Layout	22
Five conventions that explain most of the code	22
Running it	22
Where to start reading	22
Known limits in Phase 1	23

Compass is an agentic workspace: one Angular 21 front end over a FastAPI backend, six modules sharing one identity, one model gateway and one storage layer. Phase 1 ships 136 HTTP endpoints across ~47,600 lines of Python and ~49,800 lines of TypeScript, with every record in Azure Cosmos DB and every byte in Azure Blob Storage.

At a glance: the whole system

One signed cookie guards six modules, one model gateway and three Azure stores

Compass Phase 1 · browser to store, the path every request takes

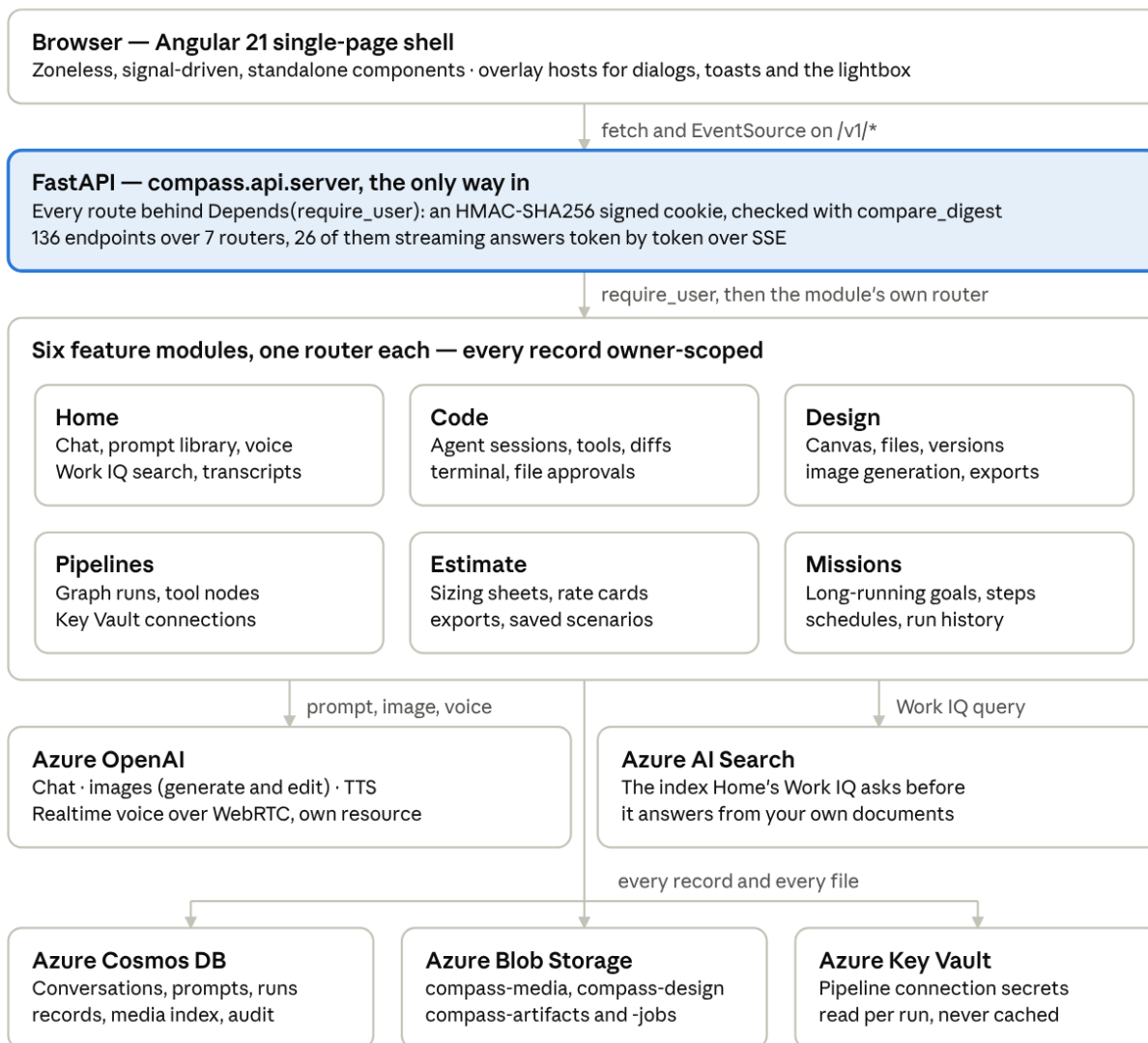


Figure 1 — The whole system: browser, API gate, six modules, model gateway and the three Azure stores.

Read it downward. The browser is one Angular application; every call it makes enters FastAPI at /v1/*, and the accented band is the only door — no route in Compass is reachable without the signed cookie. Below the door sit the six modules, each a router of its own, each scoping every record it reads to the person who owns it. The two boxes under them are the only services Compass asks questions of; the three at the foot are the only places it keeps anything. Nothing in Phase 1 persists to the local disk except the pipeline secrets file, which is why Key Vault appears at the bottom right.

The Azure infrastructure

Every Azure service Compass uses — and the one call the browser makes itself

Phase 1 runs the API on one machine; everything it keeps or asks lives in Azure

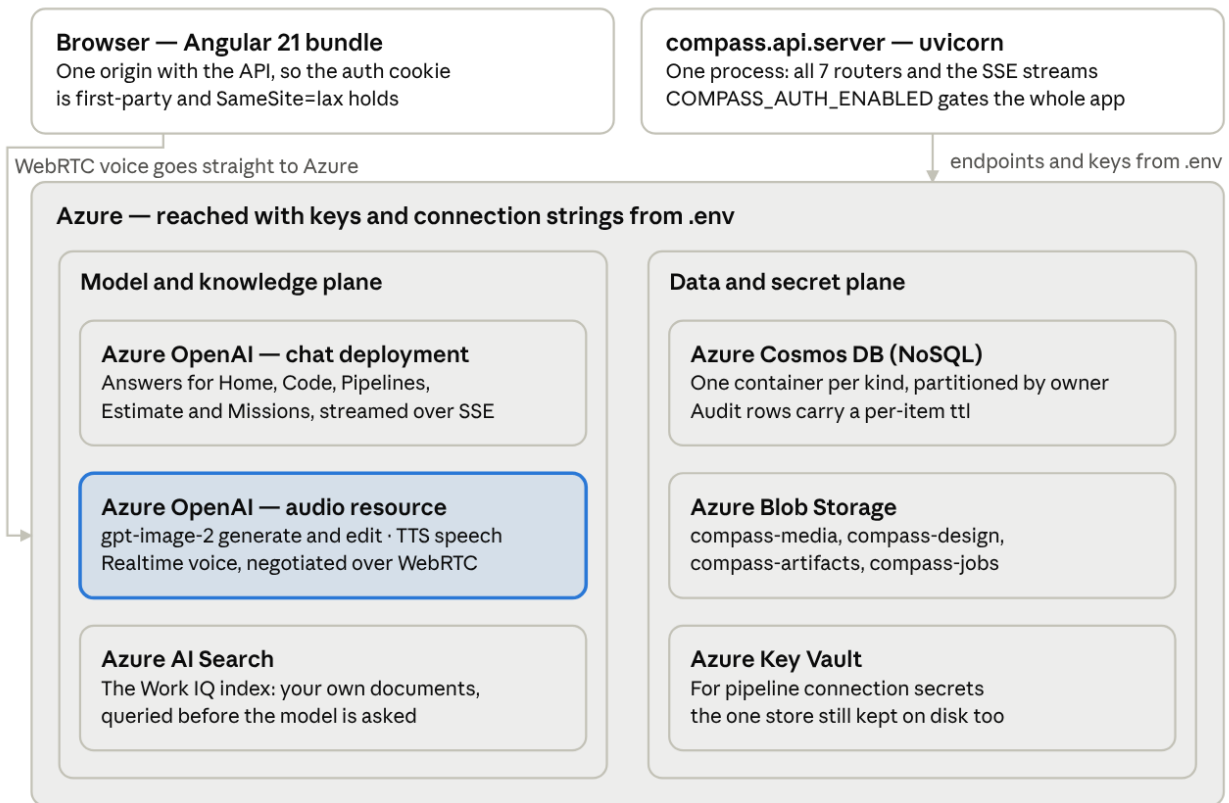


Figure 2 — Azure footprint: two runtime pieces, six services, and the one path that skips the server.

Six Azure services, two planes. Everything in the left plane is something Compass **asks**; everything in the right plane is something it **keeps**. The server holds every credential — they are read from `.env` at start-up and never reach the browser.

The highlighted box is the single exception to that rule, and it is worth understanding. Realtime voice cannot be proxied: the browser has to open a WebRTC peer connection to the Azure OpenAI audio resource itself. The server still holds the real key; it mints a short-lived ephemeral credential, hands only that to the page, and the page negotiates directly. Note also that image generation and speech live on the *audio* resource rather than the chat one — `gpt-image-2` is deployed there, which is a deployment fact worth knowing before anyone tries to point it at the chat endpoint.

Key Vault is the one line in this picture that is not yet finished. It is wired for pipeline connection secrets, but those secrets are still also written to `data/secrets/connections.json` on the machine running the API. Closing that gap needs a vault URL, and it is the last item standing between Phase 1 and a server that keeps nothing locally.

Sign-in and the request path

The cookie is minted once and re-checked on every call that follows

Sign-in, then one streamed answer, end to end

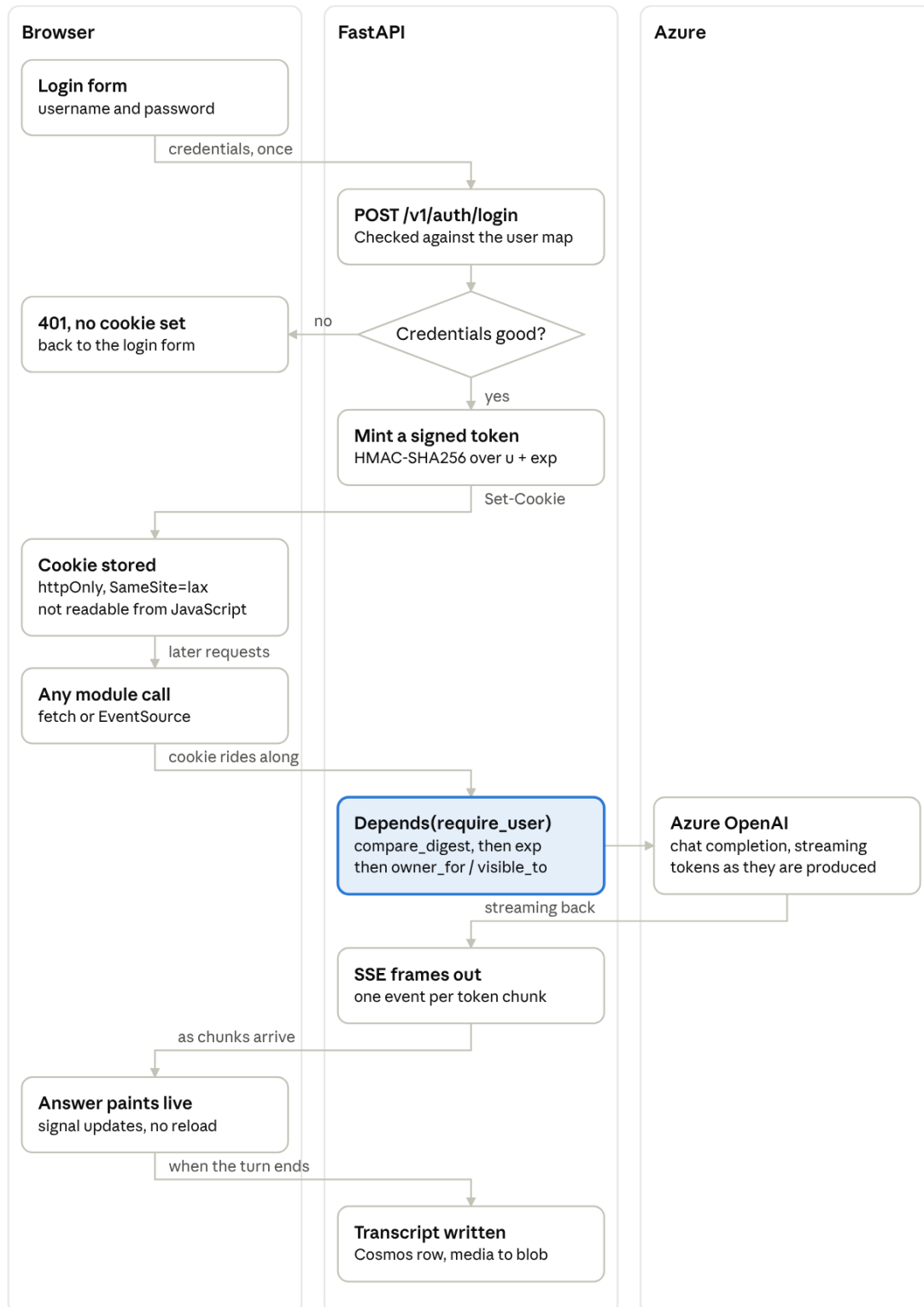


Figure 3 — Sign-in to a streamed answer, across three lanes.

The top third happens once a session; everything below **Cookie stored** happens again on every single call, and that repetition is the point of the picture.

The token itself is `base64(payload).base64(signature)`, where the payload is `{"u": username, "exp": ...}` and the signature is HMAC-SHA256 over it using the server's secret. Nothing in the cookie is encrypted — anyone holding it can read the username — but nothing in it can be *changed*, because `require_user` recomputes the signature with `hmac.compare_digest` and rejects a mismatch in constant time. A tampered or expired cookie takes the same road as a wrong password: 401, and the front end routes back to the login form.

The highlighted step is where authentication becomes authorisation. `require_user` returns a username; the module router then narrows every query to that person through `owner_for`, `visible_to` and `owned`. Those three live in one file on purpose — scoping is the kind of rule that rots when it is copied into six modules, and a single implementation is the only way a reviewer can confirm that Home, Code, Design, Pipelines, Estimate and Missions all enforce it the same way. They deliberately fail *open* on an empty owner, which is what keeps a single-user deployment with `COMPASS_AUTH_ENABLED=0` working without a parallel code path.

The last step is easy to overlook and matters for reading the rest of this document: the transcript is written **after** the stream finishes, not during it. A turn interrupted mid-stream leaves no row behind.

Home (Chat)

Home is the tool-light conversation surface: ask, brainstorm, search the web, read a link, draw a picture or cut a film. It deliberately cannot touch your files — that is Code's job — and it runs in its own router (`/v1/chat`) with its own transcript store, so a fault in the agent console cannot reach it.

The turn. `POST /v1/chat/sessions/{id}/messages` opens a Server-Sent Events stream. Tokens arrive as they are produced; tool calls, cited sources and errors arrive as named events on the same stream. `abort` cancels mid-turn, `regenerate` replays the last question, `edit` rewrites a question and re-runs from there, and `fork` copies a thread up to a chosen message so a different direction can be explored without losing the original.

The four tools. Home's agent gets `memory` (durable notes about you), `web_fetch` (read a URL), `make_video` (cut photos and clips into a film with ffmpeg) and, when an image deployment is configured, `generate_image` and `edit_image` on gpt-image-2. A system clause makes the distinction that matters: when somebody asks it to *make* something it makes the thing, rather than handing back the code that would make it.

The prompt library is the part with the most machinery, and it is served by three separate agents so that each answers the question it is actually good at:

Agent	Route	Reads	Answers with
<code>sharpen</code>	<code>POST /v1/chat/prompts/sharpen</code>	the conversation a prompt came from	one merged prompt + the turn numbers used
<code>compose</code>	<code>POST /v1/chat/prompts/compose</code>	the same conversation	the merged prompt and a verdict on every message — in or out, why, and the line it contributes
<code>refine</code>	<code>POST /v1/chat/prompts/refine</code>	the saved prompt alone	a clearer rewrite, the changed phrases marked, in one of three directions (clearer, shorter, more specific)

Saving from a message shows its working: a tray of every message in the session, each with the reason it was kept or dropped, and hovering a row lights the sentence it contributed. Unticking one re-joins the rest instantly, because the lines are already in hand. Editing a saved prompt swaps that offer for Refine, since a prompt in the library has no conversation behind it.

Voice uses Azure OpenAI Realtime over WebRTC. `POST /v1/chat/voice/session` mints a short-lived ephemeral key so the browser connects directly to Azure — audio never passes through Compass.

Work IQ is optional retrieval over Azure AI Search. When it is on, the answer carries a `work_iq_sources` event and the citations render under the reply.

Attachments are kept, not just read: photos, clips and recordings are written to blob under `chat/<session>/` so a film asked for three turns later can be cut from them.

Code (Agent Console)

Code is the module with hands. An agent runs against a real folder on disk, reads and edits files, runs shell commands, drives a browser, and asks permission before anything it cannot take back.

The tool catalogue — 18 tools. `file_read`, `file_write`, `file_edit`, `glob`, `grep` for the filesystem; `bash` and `bash_output` for a persistent shell whose working directory survives between calls; `browser` and `screenshot` for a real Chromium the agent can see; `web_fetch`, `memory`, `todo_write`, `ask_user`, `consult`, `agent` (a subagent in its own sidechain), `find_tools` (used when the catalogue is too large to describe in full each request), and `generate_image` / `edit_image`.

Permission is a gate, not a setting. Every tool declares whether it is read-only. A write, a shell command or a browser navigation goes through a `PermissionBroker` with one of three policies — `interactive` (the person is asked, and the request streams to the UI as a card), `auto_grant`, or `auto_deny`. `POST /v1/sessions/{id}/permissions/{request_id}` carries the answer back into the waiting turn. Shell commands additionally pass a static analyser (`bash_security.py`) before they are offered.

The sandbox. On macOS and Linux the agent's shell runs inside an OS sandbox scoped to the session's workspace root. `COMPASS_SANDBOX=0` turns it off; on Windows, where no boundary exists, Compass warns once rather than pretending.

Workspaces are the folders Compass is allowed to work in. The module registers them (`POST /v1/workspaces/folder`), clones from GitHub, lists and searches files, shows `git status` and diffs, discards changes, opens the folder in VS Code and raises pull requests. Removing a workspace makes Compass forget the folder — nothing inside it is deleted.

Routines are scheduled agent runs: a saved prompt, a workspace and a cron-like trigger. Each run is recorded, and a finished run can fire a native OS notification. `GET /v1/routine-runs/recent` is what the shell polls to surface them.

Background tasks are long-running processes the agent spawned — a dev server, a test watcher. They outlive the turn that started them, stream their logs to `GET /v1/background-tasks/{id}/logs`, and can be stopped from the UI.

Artifacts. When the agent produces a self-contained document — a diagram, an SVG, an interactive page — it is rendered in a side panel rather than dumped into the transcript, inside a sandboxed iframe.

Design

Design turns a brief into a finished HTML document you can look at, edit in place and export. Fourteen templates set the brief the model is given: `flier`, `slides`, `mockups`, `mobile`, `email`, `animation`, `blank`, `wireframe`, `diagram`, `document`, `resume`, `research`, `object3d` and `colortype`.

The canvas is a sandboxed iframe (`sandbox="allow-scripts allow-popups"`) fed by `srcdoc`, with an editor agent injected into the preview and never into the stored markup. The opaque origin is a real constraint, not a detail: a cookie is withheld from requests made inside it, which is why generated pictures are served from an unauthenticated route whose names are 128-bit random ids.

Generated artwork. Seven templates draw their own images with gpt-image-2 — `flier`, `slides`, `mockups`, `mobile`, `email`, `animation`, `blank`. Seven deliberately do not: a `wireframe`, a `diagram`, a `document`, a `résumé`, `research`, a 3D object and a colour study are all worse with a photograph in them. The model writes `data-draw="..."` placeholders into its markup and `artwork.fill()` replaces each with a drawn picture, capped at six per design and rendered at medium quality — measured on this deployment at low 24s, medium 37s, high 112s, and six at high would add two minutes to a document that already takes minutes.

Design systems are reusable looks: a palette, type and tokens, imported from pasted text, a stylesheet, a URL or a repository, and distilled into a system a project can be based on.

Pages, versions and files. A project holds pages; every save keeps a version that can be restored; and each project has its own folder of assets, scraps and uploads — now written through to blob so the project opens complete on any machine.

Comments are pinned to elements in the rendered design, and `POST /v1/design/projects/{id}/element` edits one element in place rather than regenerating the page.

Export produces a self-contained HTML file; thumbnails are rendered server-side with headless Chromium and cached.

Pipelines

Pipelines wires Compass's own capabilities together and runs them on a trigger — a directed graph of nodes with typed ports, executed by an engine that records every run.

Node types fall into four families:

Family	Nodes	Purpose
Flow	<code>flow.start</code> , <code>flow.if</code> , <code>flow.filter</code> , <code>flow.foreach</code> , <code>flow.wait</code> , <code>flow.approval</code> , <code>flow.set_variable</code> , <code>flow.fail</code>	branching, looping, waiting, human approval
Connectors	<code>connector.http</code> , <code>gmail.trigger</code> , <code>list_messages</code> , <code>get_message</code> , <code>send</code> , <code>list_issues</code> , <code>create_issue</code> , <code>list_pulls</code>	reaching outside — HTTP, Gmail, GitHub
Database	<code>mssql.query</code> , <code>mssql.insert</code>	SQL Server read and write
Tools	any agent tool, exposed as a node	the same catalogue Code uses

Runs. `POST /v1/pipelines/{id}/run` streams progress as SSE; each node records its status, output, attempts and timing. A run can be cancelled, resumed after an approval node, or re-run a single node. History is partitioned by pipeline, so one pipeline's runs are one partition.

Connections hold the credentials a connector needs. They support OAuth sign-in with a callback route, a test call before you rely on them, and are checked for ownership on every use — a connection someone else owns is reported as absent.

The secret store is an interface with two implementations: `LocalSecretStore` (a file) and `KeyVaultSecretStore` (Azure Key Vault). It falls back to local **loudly** when Key Vault is asked for without a vault URL, because a silent downgrade to a weaker store is the failure nobody notices until it matters. This is the one piece of Compass still on local disk in the current deployment.

The builder can be generated: `POST /v1/pipelines/{id}/build` streams a model turn that assembles the graph from a description.

Capabilities are granted at edit time and checked twice — by the engine before a handler is called, and by the handler itself.

Estimate

Estimate answers two questions about a proposed system: should it be built with AI at all, and what does the first year cost. A model reads the brief and labels it; **every figure after that is computed**, which is the design decision the whole module rests on — the boundary between "a model was involved" and "arithmetic" is kept as a boundary.

Eight stages, streamed to the UI as they run so a costing that takes a minute reads as progress:

1. **classify** — read the brief, label each use case
2. **architect** — choose the delivery platform that fits
3. **feasibility** — score AI necessity, agentic fit and traditional fit
4. **cost** — development, infrastructure and tokens
5. **tokens** — project volume and model mix
6. **compare** — this against building it the standard way
7. **roi** — benefit, payback and three-year value
8. **report** — compose the document

What comes out: a verdict (*build with AI, hybrid, or build it the standard way*), a feasibility score out of 100, an itemised first-year cost, a token projection, a side-by-side comparison with the conventional build, and a payback period — with the confidence in each stated rather than implied.

Rate cards are per-owner ([PUT /v1/estimates/rate-card](#)), so a team's own day rates drive the arithmetic instead of defaults.

Exports are produced server-side as Excel and PDF, and a BRD can be generated from the same brief.

Missions

A mission is a long build that runs itself: a goal, a budget and a workspace, worked through session by session until the features are done or the money runs out. It is the only module where Compass decides what to do next without being asked each time.

The supervisor loop. `next_step()` reads the mission's state and chooses: plan, build, or review. Each session runs with a persona, and the split is enforced in code rather than in the prompt — a **builder** gets `mission_claim` and never sees `mission_verdict`, so it cannot mark its own work done however convinced it is; a **reviewer** gets the verdict tool and no way to write a file at all, because an evaluator that can edit the code it is grading fixes the bug instead of reporting it; a **planner** writes the plan and claims nothing.

Budget. \$25 by default. Spend accumulates across sessions, and the UI projects how far the remaining budget reaches — "this budget reaches about feature 6 of 50".

Stall handling. Consecutive failures are counted and backed off exponentially. After a stall limit the mission pauses and says why, with the last error shown — in practice usually a model rate limit. *Continue anyway* overrides it.

Live progress streams over SSE (`GET /v1/missions/{id}/stream`), and a reopened browser replays what it missed rather than hanging on a stream that will produce nothing.

The workspace is never deleted. Deleting a mission removes its record and its session history; the folder it built stays exactly where it is on disk, and the deletion audit records the path so the decision is explicit rather than silent.

Where the business value is

Each module earns its place differently. The mechanism matters more than the claim: what is actually removed from somebody's week.

Module	Who it serves	What it replaces	Where the return comes from
Home	everyone	ad-hoc ChatGPT use outside the tenancy	one governed surface — the conversation, its attachments and what it drew stay inside your Azure subscription
Code	engineers	manual edit–run–test cycles	an agent that reads the repo, makes the change, runs it and raises the PR; routines do the recurring parts unattended
Design	marketing, pre-sales, product	an agency round trip or a designer's queue for a flier, deck or mock-up	a first draft in minutes, editable in place, on your own design system
Pipelines	operations	scripts nobody owns and manual handoffs	scheduled, auditable automation with approval gates and a recorded history per run
Estimate	pre-sales, delivery leads	a spreadsheet rebuilt per bid	a consistent, defensible costing in minutes, with the verdict and confidence stated
Missions	engineering leadership	a backlog nobody has capacity to start	a long build that proceeds unattended inside a fixed budget, with a reviewer that cannot mark its own work done

Two cross-cutting effects. First, one identity and one storage layer means the audit question — *who made this, who deleted it, where is it* — has a single answer across six modules. Second, every artefact is portable: a conversation, a design or an estimate opens on any machine the person signs into, because nothing of record lives on the box it was made on.

Where every piece of data lives

Two rules decide the split: **a record goes to Cosmos, a byte goes to blob**. Anything over roughly 2 MB is spilled to blob with a reference left in the document, because that is Cosmos's item limit.

Azure Cosmos DB — database *compass*

Container	Partition key	Holds	Written by
<i>chat</i>	<i>sessionId</i>	Home transcripts and thread metadata	Home
<i>transcripts</i>	<i>sessionId</i>	Code agent transcripts	Code
<i>transcripts-meta</i>	<i>owner</i>	Code session titles, ownership, model, mode	Code
<i>catalog</i>	<i>owner</i>	the small owner-scoped records: media index, prompts, prompt order, routines, missions, workspaces	all modules
<i>designs</i>	<i>owner</i>	design projects and systems	Design
<i>pipelines</i>	<i>owner</i>	pipeline definitions and connections	Pipelines
<i>runs</i>	<i>parentId</i>	pipeline and routine run history, partitioned by its parent	Pipelines, Code
<i>estimates</i>	<i>owner</i>	estimates and per-owner rate cards	Estimate
<i>users</i>	<i>id</i>	accounts	Auth
<i>audit</i>	<i>owner</i>	one row per deletion, expired by a per-item TTL	all modules

Azure Blob Storage

Container	Prefix	Holds
<i>compass-media</i>	<i>generated/</i>	pictures drawn or edited by gpt-image-2
<i>compass-media</i>	<i>screenshots/</i>	what the Code agent's browser saw
<i>compass-media</i>	<i>chat/<session>/</i>	a thread's uploads and the films cut from them
<i>compass-media</i>	<i>messages/<session>/</i>	message content spilled over the Cosmos item limit
<i>compass-design</i>	<i>pages/, versions/</i>	design markup and version history
<i>compass-design</i>	<i>files/<project>/</i>	a design's own assets, scraps and uploads
<i>compass-artifacts</i>	—	tool-result spills
<i>compass-jobs</i>	—	long-running job state

Still on local disk, by design

The agent's workspaces (*data/workspaces/*) — your actual code, which belongs on the machine. Everything else local is either a cache that rebuilds from blob on demand, or *data/secrets/connections.json*, the one record not yet in the cloud: Key Vault is implemented and needs only a vault URL.

The retention rule

Deleting removes the record **and** the bytes — the Cosmos row, the blob, and the working copy on disk. What survives is one audit row saying who deleted what, from which module, which session it belonged to and what went with it — never the content. Those rows carry a `ttl` and Cosmos expires them itself, so retention needs no sweep. `COMPASS_AUDIT_RETENTION_MONTHS` sets it; the default is three months, `0` keeps them.

API reference

136 endpoints. Every one except the two byte routes noted below requires the session cookie via `Depends(require_user)`. The **Called from** column names the method in `frontend/src/app/compass-api.service.ts`, which is the only place the front end issues HTTP — components call that service, never `HttpClient` directly.

Auth — `compass/common/auth.py`

Method & path	Purpose	Store	Called from
POST <code>/v1/auth/login</code>	exchange credentials for the cookie	users	<code>AuthService.login</code>
POST <code>/v1/auth/logout</code>	clear the cookie	—	<code>AuthService.logout</code>
GET <code>/v1/auth/me</code>	who am I	users	<code>AuthService.restore</code>
POST <code>/v1/auth/me/name</code>	set display name	users	account menu
GET <code>/v1/auth/users</code>	list accounts	users	admin surface

Home — `compass/home/routes.py`, prefix `/v1/chat`

Method & path	Purpose	Store	Called from
GET <code>/sessions</code>	the sidebar list	chat	<code>listChatSessions</code>
POST <code>/sessions</code>	start or resume a thread	chat	<code>createChatSession</code>
GET <code>/sessions/{id}/transcript</code>	paged history	chat + blob spills	<code>chatTranscript</code>
POST <code>/sessions/{id}/messages</code>	SSE turn	chat, blob	<code>streamChatMessage</code>
POST <code>/sessions/{id}/regenerate</code>	redo the last answer	chat	<code>streamChatRegenerate</code>
POST <code>/sessions/{id}/edit</code>	rewrite a question, re-run	chat	<code>streamChatEdit</code>
POST <code>/sessions/{id}/fork</code>	branch the thread	chat	<code>forkChatSession</code>
POST <code>/sessions/{id}/abort</code>	cancel mid-turn	—	<code>abortChat</code>
PATCH <code>/sessions/{id}</code>	rename, star	chat	<code>patchChatSession</code>
DELETE <code>/sessions/{id}</code>	delete thread + media + index rows	chat, blob, catalog, audit	<code>deleteChatSession</code>
GET <code>/sessions/{id}/media/{file}</code>	serve an upload	blob	<code> / <video> src</code>
POST <code>/sessions/{id}/voice-turn</code>	a spoken turn	chat	voice mode
GET <code>/prompts</code>	the library + its order	catalog	<code>savedPrompts</code>
POST <code>/prompts</code>	save one	catalog	<code>savePrompt</code>
PATCH <code>/prompts/{id}</code>	edit one	catalog	<code>editSavedPrompt</code>
DELETE <code>/prompts/{id}</code>	delete one	catalog, audit	<code>deleteSavedPrompt</code>
PUT <code>/prompts/order</code>	the arrangement	catalog	<code>setPromptOrder</code>
POST <code>/prompts/sharpen</code>	merge a thread into one prompt	none — generation only	<code>sharpenPrompt</code>

Method & path	Purpose	Store	Called from
POST /prompts/compose	the same, plus a verdict per message	none	composePrompt
POST /prompts/refine	rewrite a saved prompt	none	refinePrompt
GET /work-iq	is retrieval configured	—	workIqStatus
GET /voice	is realtime configured	—	voiceStatus
POST /voice/session	mint an ephemeral WebRTC key	—	voiceSession
GET /favicon	cached site icon for a citation	local cache	citation strip

Code — compass/code/routes.py

Method & path	Purpose	Store	Called from
GET /v1/sessions	session list	transcripts -meta	listSessions
POST /v1/sessions	new session	transcripts -meta	createSession
GET /v1/sessions/{id}/transcript	history	transcripts + blob	transcript loader
POST /v1/sessions/{id}/messages	SSE agent turn	transcripts , blob	streamMessage
POST /v1/sessions/{id}/messages/{uuid}/edit	edit and re-run	transcripts	streamEdit
POST /v1/sessions/{id}/regenerate	redo	transcripts	streamRegenerate
POST /v1/sessions/{id}/permissions/{req}	answer a permission prompt	—	resolvePermission
POST /v1/sessions/{id}/questions/{req}	answer ask_user	—	answerQuestion
POST /v1/sessions/{id}/suggest	next-step suggestions	—	suggestNext
POST /v1/sessions/{id}/fork · abort	branch · cancel	transcripts	forkSession · abort
PATCH · DELETE /v1/sessions/{id}	rename · delete + purge media	meta, blob, audit	updateSession · deleteSession
GET /v1/workspaces · POST /v1/workspaces/folder	list · register a folder	catalog	listWorkspaces · addFolderWorkspace
DELETE /v1/workspaces/{id}	forget the folder (files untouched)	catalog, audit	deleteWorkspace
GET /v1/workspaces/{id}/files · file · files/search	browse · read · search	disk	listFiles · readFile · searchFiles
GET /v1/workspaces/{id}/git · diff	status · diff	git	gitStatus · gitDiff
POST /v1/workspaces/{id}/discard · pr · open-in-vscode	revert · raise PR · open editor	git, GitHub	discardChanges · createPr · openWorkspaceInVsCode
POST /v1/github/clone · GET /v1/github/repos	clone · list repos	disk, GitHub	githubClone · githubRepos
GET · POST · PATCH · DELETE /v1/routines...	scheduled runs CRUD	catalog, runs	routines, createRoutine, updateRoutine, deleteRoutine

Method & path	Purpose	Store	Called from
POST /v1/routines/{id}/run · GET /v1/routine-runs/recent	run now · poll finished	runs	runRoutineNow · recentRoutineRuns
GET /v1/background-tasks (+ logs, stop, clear)	spawned processes	in-process	backgroundTasks, backgroundTaskLogs, stopBackgroundTask
POST /v1/pick-folder	native folder picker	—	pickFolder

Design — compass/design/routes.py

Method & path	Purpose	Store	Called from
GET /v1/design/templates	the 14 templates	—	designTemplates
GET · POST /v1/design/projects	list · create	designs	designProjects · createDesign
GET · PATCH · DELETE /v1/design/projects/{id}	open · rename · delete	designs, blob, audit	designProject · patchDesign · deleteDesign
POST /v1/design/projects/{id}/generate	SSE generate the design, draw its artwork	designs, blob, catalog	design chat
POST /v1/design/projects/{id}/html	save edited markup	blob	saveDesignHtml
POST /v1/design/projects/{id}/element	edit one element in place	blob	editElement
GET · POST · DELETE .../pages	the project's pages	blob	designPages, addDesignPage, deleteDesignPage
GET .../versions · POST .../versions/{v}/restore	history · restore	blob	designVersions · restoreDesignVersion
GET · POST · DELETE .../files (+ files/read)	the project's own folder	blob + disk cache	designFiles, writeDesignFile, deleteDesignFile
POST · DELETE .../comments	pin and remove comments	designs	addDesignComment · deleteDesignComment
GET .../thumbnail · .../export HTML	rendered thumb · standalone HTML	cache · blob	designThumbUrl · designExportUrl
GET · POST · DELETE /v1/design/systems...	design systems CRUD, setup, duplicate, usage	designs	designSystems, createDesignSystem, setUpDesignSystem
POST /v1/design/clarify · /attach	sharpen a brief · keep an attachment	blob	clarifyDesign

Pipelines — compass/pipelines/routes.py

Method & path	Purpose	Store	Called from
GET · POST /v1/pipelines	list · create	pipelines	pipelines · createPipeline
GET · PATCH · DELETE /v1/pipelines/{id}	open · save · delete + its runs	pipelines, runs, audit	getPipeline · savePipeline · deletePipeline
POST /v1/pipelines/{id}/run	SSE execute the graph	runs	runPipeline

Method & path	Purpose	Store	Called from
POST /v1/pipelines/{id}/nodes/{node}/run	run one node	runs	runPipelineNode
POST /v1/pipelines/{id}/validate	what is wrong with this graph	—	validatePipeline
GET · POST · DELETE /v1/pipelines/{id}/build	SSE build the graph from a description	pipelines	streamBuild · resetBuild
GET /v1/pipelines/{id}/runs · GET /v1/pipeline-runs/{run}	history · one run	runs	pipelineRuns
POST /v1/pipeline-runs/{run}/cancel · resume	stop · continue past approval	runs	resumePipelineRun
GET · POST · PATCH · DELETE /v1/pipeline-connections	connection CRUD	pipelines + secret store	pipelineConnections, createConnection, deleteConnection
POST .../{id}/test · sign-in · GET .../oauth/callback	verify · OAuth	secret store	testConnection · signInConnection
GET /v1/pipelines/node-types · /v1/pipeline-connection-types	the palettes	—	connectionTypes

Estimate — compass/estimate/routes.py

Method & path	Purpose	Store	Called from
GET · POST /v1/estimates	list · create	estimates	EstimateApi.list · create
GET · DELETE /v1/estimates/{id}	open · delete	estimates, audit	EstimateApi.get · remove
POST /v1/estimates/stream	SSE the eight stages	estimates	estimate runner
POST /v1/estimates/preview · draft · brd	platform preview · draft · generate a BRD	—	estimate wizard
GET /v1/estimates/{id}/export/excel · pdf	downloads	—	export buttons
GET · PUT /v1/estimates/rate-card	per-owner day rates	estimates	rate-card editor
GET /v1/estimates/catalog	archetypes and defaults	—	wizard

Missions — compass/missions/routes.py, **prefix** /v1/missions

Method & path	Purpose	Store	Called from
GET · POST	list · create	catalog	missions · createMission
GET /{id}	one mission with features	catalog	mission
POST /{id}/run · GET /{id}/stream	SSE start · watch	catalog, transcripts	streamMission
POST /{id}/abort · DELETE /{id}	stop · delete (workspace kept)	catalog, audit	abortMission · deleteMission

Shared — `compass/common/routes.py`

Method & path	Purpose	Store	Called from
GET <code>/healthz</code>	liveness, models, capabilities	—	<code>health</code> on boot
GET <code>/v1/models</code>	deployments and effort ladders	—	model picker
GET <code>/v1/media</code>	everything this person has made	<code>catalog</code>	media browser
GET <code>/v1/media/generated/{name}</code>	serve a drawn picture — unauthenticated by design	blob	<code></code> in chat and the design canvas
GET <code>/v1/screenshot-cache/{id}</code>	serve an agent screenshot — auth + ownership	blob	<code></code> in the Code transcript
POST <code>/v1/screenshot</code>	capture a URL	blob	<code>screenshot</code>
GET · POST · PATCH · DELETE <code>/v1/memory</code>	durable notes about the user	<code>catalog</code>	<code>listMemory</code> , <code>patchMemory</code> , <code>deleteMemory</code>
GET <code>/v1/recap</code>	what happened in the last N days	several	<code>recap</code>
GET <code>/v1/customize</code>	per-user preferences	<code>users</code>	<code>customize</code>
POST <code>/v1/speech</code>	text to speech	—	<code>synthesizeSpeech</code>

The two byte routes are deliberately different. `/v1/media/generated/{name}` has no auth because it is read by the design canvas, a sandboxed iframe with an opaque origin that a SameSite cookie is withheld from; its names are 128-bit random ids, so the URL is the credential — unguessable but copyable, and nothing should be put there that only the login protects. `/v1/screenshot-cache/{id}` has both auth *and* an ownership check, because a screenshot is whatever the agent's browser was looking at and its id is only twelve hex characters.

Identity, ownership and deletion

The token. `POST /v1/auth/login` compares the password with `hmac.compare_digest` — constant time — and returns a signed token as an `httpOnly` cookie: `base64(payload).base64(hmac_sha256(payload, secret))`, where the payload is `{"u": username, "exp": unix_seconds}`. There is no server-side session; `require_user` verifies the signature and expiry on every request and is a dependency on all 136 stateful routes. Accounts come from `COMPASS_AUTH_USERS` as one comma-separated list — `alice:pw1,bob:pw2`. That variable **replaces** the whole account map; a second variable with another name is not read.

The ownership rule lives in one file, `compass/common/ownership.py`, so it cannot drift between modules. Two functions: `owner_for(user)` stamps a record on creation, and `visible_to(user, owner)` decides a read. It **fails open on an empty owner** — a record written before ownership existed is visible to everyone. That is deliberate and right for a single-account install, where the alternative looks exactly like data loss, and wrong the moment a second person signs in. The migration scripts in `scripts/` exist to close that gap by naming the owner, rather than by changing the rule.

One module is stricter: the **prompt library** matches the owner exactly, because a saved prompt is written in the first person about somebody's own day, and showing one to a stranger is a different kind of wrong from showing them a shared pipeline.

Not yours reads as not there. Every per-record route answers 404 rather than 403 when a record belongs to someone else. Whether a given id exists is itself something a person who cannot see it has no business learning.

Deleting cascades. Removing a conversation takes its transcript, its uploads, the pictures drawn in it, the screenshots taken in it and their index rows — the Cosmos rows, the blobs and the working copy on disk. Deleting a pipeline takes its whole run history. Deleting a mission leaves the workspace it built on disk, and says so in the audit row.

And it is recorded. One row per deletion in the `audit` container, partitioned by owner: who, what, which module, which session, when, and a tally of what went with it — never the content, because the content is the thing that was deleted. Writing the note can never cost a deletion: every call swallows its own failures, so somebody who asked for something to be removed has it removed whether or not the log was reachable.

In the browser, signing out clears everything the shell holds on someone's behalf before the next person arrives, and the same clearing runs on sign-in, because a session that expires never calls sign-out. Three per-browser preferences remain in `localStorage` — theme, notification sound, and whether the pipeline builder panel is open. They hold no data.

For a new developer

Layout

```
compass/
  api/server.py      the FastAPI app – routers, lifespan, middleware
  common/           72 files: auth, config, ownership, audit, persistence,
                    the model gateway, the shared tool base, media index
  home/             Chat: engine, prompts, media, video, vision, favicons
  code/             Agent console: engine, routines, workspaces, tools/
  design/           projects, systems, markup, artwork, export, files
  pipelines/        engine, nodes/, store, secrets, credentials, oauth
  estimate/         pipeline (8 stages), engine, catalog, exports
  missions/         supervisor, engine, runner, personas, artifacts
  frontend/src/app/ Angular 21, standalone + signals, zoneless
  scripts/          one-off migrations, each with a --apply gate
```

Five conventions that explain most of the code

1. **One file owns a rule.** Ownership lives in `ownership.py`, deletion notes in `audit.py`, the tool contract in `tools/base.py`. If a rule appears twice, that is a bug waiting to happen.
2. **Comments say *why*, not *what*.** The codebase explains the decision and often the failure that caused it. When you change behaviour, update the reasoning beside it — a stale *why* is worse than none.
3. **Best-effort never blocks the main job.** Index rows, audit notes and blob mirrors all swallow their own failures. A picture whose index row failed is still drawn, stored and shown.
4. **Optimistic UI with rollback.** The row leaves the list before the request does; on failure the previous state is restored exactly and the error names the server's own reason.
5. **Migrations are scripts, not startup code.** Everything in `scripts/` dry-runs by default and needs `--apply`. None of them delete the source.

Running it

```
# backend – reads .env from the repo root
uvicorn compass.api.server:app --port 8000

# frontend – proxies /v1 to :8000
cd frontend && npm start          # http://localhost:4200
```

The variables that decide behaviour: `COMPASS_STORAGE_BACKEND` (`local` or `cosmos`), `AZURE_COSMOS_ENDPOINT` / `_KEY`, `AZURE_STORAGE_CONNECTION_STRING`, `AZURE_OPENAI_ENDPOINT` / `_KEY` / `_DEPLOYMENT`, `COMPASS_AUTH_ENABLED` and `COMPASS_AUTH_USERS`. Feature flags `COMPASS_PIPELINES`, `COMPASS_ESTIMATE` and `COMPASS_MISSIONS` gate whole routers — a module that is off never registers its routes.

Where to start reading

Follow one turn end to end: `frontend/src/app/home-chat/home-chat.ts` → `compass-api.service.ts` → `compass/home/routes.py` → `compass/home/engine.py` → `compass/common/gateway/azure_client.py`. That path touches the streaming protocol, the tool loop, the model gateway and the transcript store — which is most of what the other five modules also do.

Known limits in Phase 1

- Pipeline connection secrets are still a local file; the Key Vault backend is written and needs a vault URL.
- Design's generated artwork has not been exercised end to end — no stored design yet carries a drawn picture, and the sandboxed canvas may not load one when it does.
- Ownership fails open on legacy records, by design. New installs are unaffected; migrated ones need the `scripts/claim_unowned_*.py` pass.